Análise de Algoritmos Aplicada ao Cadastro de Bolsistas da CAPES

Relatório 3 — Laboratório 10/09

Wilton Rodrigues do Carmo Filho
Grupo 17
Análise de Algoritmos (GPE02M60042) — Universidade Católica de Brasília

15 de outubro de 2026

Repositório Git público: `https://github.com/SEU-USUARIO/lab-bolsistas-capes`
Contém o código-fonte em C, o arquivo `bolsistas-2025.csv`, os arquivos de entrada dos testes, os CSV gerados nos experimentos, as saídas do programa, os scripts, as instruções de compilação e de execução (`README.md`) e a fonte deste relatório.

Conteúdo

1 Introdução **2**

2 Descrição da solução implementada **2**

3 Explicação do funcionamento do programa **2**

4 Estruturas de dados utilizadas **3**

5 Análise teórica do algoritmo **3**

6 Dedução da função $C(n)$ **4**

7 Demonstração por indução matemática **5**

8 Metodologia experimental **5**

9 Tabelas e gráficos dos experimentos **6**

10 Comparação entre os resultados teóricos e empíricos **8**

11 Discussão dos resultados **8**

12 Conclusão **9**

A Código-fonte (`src/copiador.c`) **9**

1 Introdução

A CAPES recebe solicitações de cadastro de bolsistas em arquivos CSV que precisam ser incorporadas ao cadastro principal sem gerar registros duplicados. Este trabalho implementa em C um programa, chamado `copiador`, que copia os registros de um CSV de origem para um CSV de destino ignorando qualquer registro cujo campo *Nome* já exista no destino, e que conta o número de comparações entre nomes realizadas pela busca de redundância.

O objetivo central, porém, é a análise do algoritmo. O relatório (i) deduz uma função $C(n)$ para o número de comparações necessárias para inserir n nomes distintos, (ii) demonstra por indução matemática que a expressão deduzida está correta, (iii) classifica o algoritmo assintoticamente, (iv) mede $C(n)$ experimentalmente com o contador implementado e (v) confronta as previsões teóricas com as medições.

2 Descrição da solução implementada

O programa recebe, pela linha de comando, o CSV de origem, o CSV de destino e o CSV de estatísticas:

```
./copiador bolsistas-2025.csv cadastro.csv estatisticas.csv
```

As regras implementadas seguem o enunciado:

- **Critério de redundância:** somente o campo *Nome* (primeiro campo) é comparado, por igualdade exata de texto (`strcmp`, sem normalização de caixa ou de espaços). Duas linhas com o mesmo Nome e demais campos diferentes são redundantes.
- **Cópia:** um registro não redundante é copiado *inteiro* (todos os campos originais) para o final do destino.
- **Redundância:** para um registro redundante o programa escreve na saída padrão `ERRO: nome redundante encontrado: <nome>`, não o copia e continua o processamento.
- **Cabeçalhos:** se o destino não existe ou está vazio, o programa cria o cabeçalho `Nome,Modalidade,Nivel,Agencia`. A primeira linha da origem, quando é o cabeçalho, não é copiada como registro.
- **Estatísticas:** ao final, uma linha `nomes_inseridos,comparacoes` é anexada ao arquivo de estatísticas (o cabeçalho é escrito se o arquivo é novo).

Convenção de contagem. Cada vez que o Nome de um registro da origem é comparado com o Nome de uma linha do destino, o contador é incrementado em uma unidade. A busca é sequencial e percorre *todas as linhas do arquivo de destino*, e o cabeçalho é uma linha desse arquivo (seu campo Nome vale `Nome`). Portanto o cabeçalho participa da busca e conta como uma comparação. Essa convenção é a que reproduz o exemplo de saída do enunciado (10 nomes $\rightarrow$ 55 comparações, 20 $\rightarrow$ 210 etc.), como mostrado na Seção 5.

3 Explicação do funcionamento do programa

A execução segue estes passos:

1. Carrega o arquivo de destino, se existir, para um vetor com o Nome de cada linha, na mesma ordem em que aparecem no arquivo. Se o destino não existe ou está vazio, escreve o cabeçalho e o registra como primeira linha do vetor.

2. Abre a origem e lê uma linha por vez. Linhas em branco são ignoradas.
3. Extrai o Nome (primeiro campo; aceita campo entre aspas).
4. Executa a busca sequencial do Nome no vetor, incrementando o contador a cada comparação.
5. Se encontrou: imprime a mensagem de erro e segue para a próxima linha. Se não encontrou: anexa a linha completa ao destino, acrescenta o Nome ao vetor e incrementa `inseridos`.
6. Ao final, anexa `inseridos,comparacoes` ao arquivo de estatísticas.

O núcleo do algoritmo e da instrumentação é a função abaixo (trecho de `src/copiador.c`):

```
static unsigned long long comparacoes = 0;

static int nome_existe(const Cadastro *c, const char *nome)
{
    for (size_t i = 0; i < c->tamanho; i++) {
        comparacoes++;                          /* uma comparacao */
        if (strcmp(c->nomes[i], nome) == 0)
            return 1;                           /* redundante */
    }
    return 0;                                   /* nome novo */
}
```

Exemplo do enunciado. Com o destino contendo o cabeçalho e `MARIA APARECIDA SILVA,DS,M,CAPES`, e a origem contendo `MARIA APARECIDA SILVA,PQ,C,CNPq` e `ABILIO AZAMBUJA RODRIGUES FILHO,PQ,C,CNPq`, a saída padrão é `ERRO: nome redundante encontrado: MARIA APARECIDA SILVA`, o destino passa a ter 3 linhas (a de ABILIO é a nova) e as estatísticas registram `1,4`: o primeiro nome é achado na 2.ª linha do destino (2 comparações) e o segundo percorre as 2 linhas sem achar (2 comparações). Esse caso é um dos testes automatizados (`make test`).

4 Estruturas de dados utilizadas

- `Cadastro`: um vetor dinâmico de `char *` com o Nome de cada linha do destino, mais o tamanho e a capacidade. A capacidade dobra quando se esgota (`realloc`), de modo que o acréscimo é $O(1)$ amortizado. A posição i do vetor corresponde à i-ésima linha do arquivo de destino, de modo que percorrer o vetor em ordem é exatamente a busca sequencial sobre o arquivo.
- **Contador** `comparacoes`: variável global do tipo `unsigned long long`, incrementada em cada passo da busca.
- **Buffers de linha**: alocados por `getline`, sem limite fixo de tamanho de linha ou de quantidade de registros.

Uma tabela de dispersão (*hash*) reduziria o custo esperado da busca para $O(1)$, mas mudaria o algoritmo que o exercício pede para analisar (busca sequencial); por isso foi mantido o vetor. O vetor em memória evita reabrir e reler o arquivo a cada verificação, sem alterar a contagem de comparações, pois cada elemento do vetor representa uma linha do destino.

5 Análise teórica do algoritmo

Hipóteses. (H1) a verificação de redundância é uma busca sequencial; (H2) os n nomes a inserir são distintos entre si e distintos dos já existentes; (H3) o destino inicia contendo apenas o cabeçalho.

Custo de uma inserção. Antes da k-ésima inserção ($k = 1, \ldots, n$) o destino tem k linhas: o cabeçalho e os $k-1$ registros já inseridos. Pela hipótese H2 nenhuma delas é igual ao novo nome, então a busca percorre as k linhas e faz exatamente k comparações.

Questão 2: função $C(n)$. O total é a soma dos custos:

$$C(n) \;=\; \sum_{k=1}^{n} k \;=\; \frac{n(n+1)}{2} \;=\; \frac{n^2}{2} + \frac{n}{2}.$$

A dedução detalhada está na Seção 6 e a demonstração por indução na Seção 7.

Questão 1: comparações para cada n.

n	$C(n) = n(n+1)/2$
10	55
20	210
40	820
80	3.240
160	12.880

Esses valores coincidem com o exemplo de saída do enunciado.

Questão 3: complexidade assintótica. $C(n) = \frac{n^2}{2} + \frac{n}{2} \leq n^2$ para todo $n \geq 1$ (pois $\frac{n+1}{2} \leq n$), logo $C(n) \in O(n^2)$ com $c = 1$ e $n_0 = 1$. Como também $C(n) \geq \frac{n^2}{2}$, vale $C(n) \in \Theta(n^2)$. As demais operações (leitura da origem, escrita no destino) custam $O(n)$ no total e são dominadas pelas comparações. O algoritmo é, portanto, quadrático.

Questão 4: crescimento quando n dobra. Como

$$\frac{C(2n)}{C(n)} \;=\; \frac{2n(2n+1)/2}{n(n+1)/2} \;=\; \frac{2(2n+1)}{n+1} \;=\; 4 - \frac{2}{n+1},$$

o número de comparações cresce pouco menos que quatro vezes e a razão tende a 4 quando n aumenta (cerca de 3,82; 3,90; 3,95 e 3,98 nas passagens 10→20, 20→40, 40→80 e 80→160). Dobrar a entrada quadruplica o trabalho, que é o comportamento característico de um algoritmo $\Theta(n^2)$.

6 Dedução da função $C(n)$

Seja a_k o número de comparações da k-ésima inserção. Pela análise da Seção 5, $a_k = k$. Então C satisfaz a recorrência

$$C(0) = 0, \qquad C(k) = C(k-1) + k \quad (k \geq 1),$$

e, desenrolando-a,

$$C(n) = 1 + 2 + \cdots + n.$$

Escrevendo a soma duas vezes, uma na ordem crescente e outra na decrescente,

$$\begin{array}{rcccccccc} C(n) & = & 1 & + & 2 & + & \cdots & + & n \\ C(n) & = & n & + & (n-1) & + & \cdots & + & 1 \end{array}$$

e somando coluna a coluna, obtêm-se n parcelas iguais a $n+1$: $2\,C(n) = n(n+1)$, isto é,

$$\boxed{C(n) = \frac{n(n+1)}{2}}$$

7 Demonstração por indução matemática

Lema 1. *Sob as hipóteses H1–H3, a k-ésima inserção realiza exatamente k comparações.*

Demonstração. Imediatamente antes da k-ésima inserção o destino contém o cabeçalho e os $k-1$ registros já inseridos, isto é, k linhas. A busca sequencial compara o novo Nome com cada uma delas, em ordem. Por H2 nenhuma é igual, de modo que a busca não termina antes do fim e executa uma comparação por linha: k comparações. □

Pelo Lema 1, $C(n)$ satisfaz $C(0) = 0$ e $C(n+1) = C(n) + (n+1)$ para todo $n \geq 0$. Seja $P(n)$ a proposição

$$P(n): \quad C(n) = \frac{n(n+1)}{2}.$$

Teorema 1. *$P(n)$ é verdadeira para todo $n \in \mathbb{N}$, $n \geq 0$.*

Demonstração. Por indução em n.

Caso base ($n = 0$). Nenhum nome é inserido, logo $C(0) = 0$, e $\frac{0\cdot 1}{2} = 0$. Portanto $P(0)$ vale. (Verificação adicional: para $n = 1$, $C(1) = 1$, o cabeçalho, e $\frac{1\cdot 2}{2} = 1$.)

Hipótese de indução. Seja $k \geq 0$ e suponha que $P(k)$ é verdadeira, isto é, $C(k) = \dfrac{k(k+1)}{2}$.

Passo indutivo. Vamos provar $P(k+1)$. Pelo Lema 1, a $(k+1)$-ésima inserção custa $k+1$ comparações, e portanto

$$\begin{aligned} C(k+1) &= C(k) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) && \text{(hipótese de indução)} \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \\ &= \frac{(k+1)\big((k+1)+1\big)}{2}, \end{aligned}$$

que é exatamente $P(k+1)$.

Conclusão. Como $P(0)$ é verdadeira e $P(k) \Rightarrow P(k+1)$ para todo $k \geq 0$, pelo princípio da indução matemática $C(n) = \dfrac{n(n+1)}{2}$ para todo $n \geq 0$. □

8 Metodologia experimental

Ambiente. Compilador: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; sistema: Linux 6.18.44-fc-v139; execução em 15/10/2026. Compilação com `gcc -std=c11 -O2 -Wall -Wextra -pedantic`, sem avisos.

Grandeza medida. O contador de comparações do próprio programa (e não tempo de relógio). Por ser uma contagem de operações determinística, uma execução por tamanho é suficiente: repetir não altera o valor, e não há ruído de medição como haveria com tempo de execução.

Procedimento. Para cada tamanho n da série obrigatória (10, 20, 40, 80 e 160) e da série ampliada (320, 640, 1.280, 2.560 e 5.120, usada para observar o comportamento assintótico):

1. extraem-se de `bolsistas-2025.csv` os n primeiros nomes *distintos*, na ordem de aparição, formando o arquivo de entrada `tests/entrada/bolsistas-N.csv`, com $N = n$ (hipótese H2);
2. cria-se um destino contendo apenas o cabeçalho (hipótese H3);
3. executa-se `./copiador` sobre esses arquivos; a linha de estatísticas é acumulada em `resultados/estatisticas.csv` (série obrigatória) e `resultados/estatisticas-ampliado.csv` (série ampliada).

Além disso, faz-se uma execução completa com o arquivo de origem inteiro, incluindo seus nomes repetidos, para observar o efeito das redundâncias.

Validação do programa. Seis testes automatizados (`tests/run_tests.sh`) verificam: o exemplo do enunciado; destino inexistente; redundância dentro da própria origem; acúmulo de linhas no arquivo de estatísticas; $n = 50$ nomes distintos com resultado $1275 = 50 \cdot 51/2$; e arquivos com final de linha CRLF ou sem quebra de linha final.

Reprodução.

```
make && make test && make experimentos
```

Os valores deste relatório (tabelas, macros e gráfico) são gerados por `scripts/gerar_dados_relatorio.py` a partir dos CSV de `resultados/`.

9 Tabelas e gráficos dos experimentos

n	Observado	Previsto $\frac{n(n+1)}{2}$	Diferença
10	55	55	0
20	210	210	0
40	820	820	0
80	3.240	3.240	0
160	12.880	12.880	0

Tabela 1: Série obrigatória: comparações observadas, previstas pelo modelo $C(n) = n(n+1)/2$ e diferença (observado − previsto).

Execução completa com redundâncias. A Tabela 3 resume a execução com o arquivo de origem inteiro, que contém nomes repetidos.

n	$C(n)$ observado	$C(n)/n^2$	$\dfrac{C(n)}{C(n/2)}$ observada	$\dfrac{4(n+1)}{n+2}$
10	55	0,5500	—	—
20	210	0,5250	3,818	3,818
40	820	0,5125	3,905	3,905
80	3.240	0,5062	3,951	3,951
160	12.880	0,5031	3,975	3,975
320	51.360	0,5016	3,988	3,988
640	205.120	0,5008	3,994	3,994
1.280	819.840	0,5004	3,997	3,997
2.560	3.278.080	0,5002	3,998	3,998
5.120	13.109.760	0,5001	3,999	3,999

Tabela 2: Todos os tamanhos medidos: $C(n)/n^2$ e razão entre medições consecutivas (cada n é o dobro do anterior), comparada ao valor teórico $4(n+1)/(n+2)$.

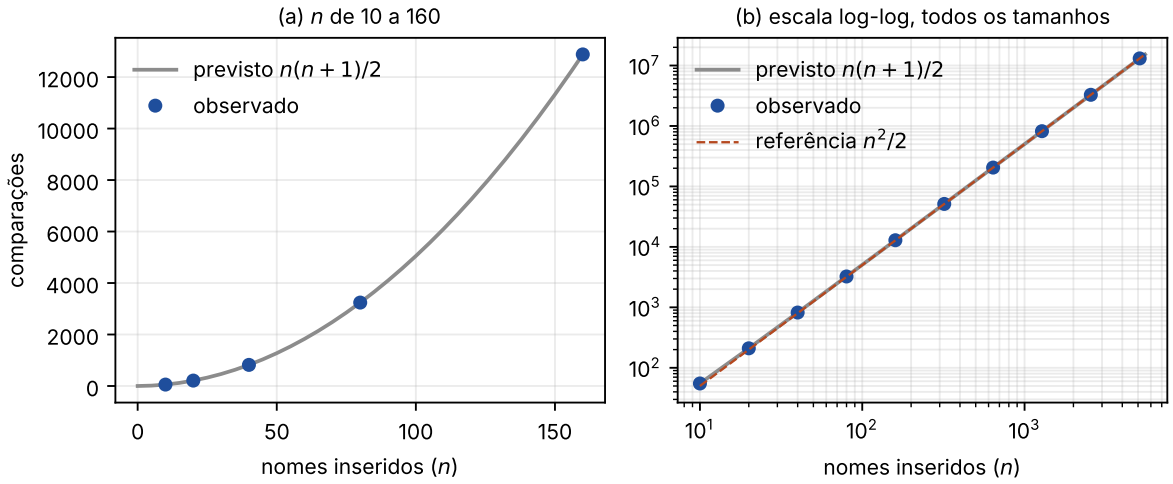


Figura 1: Comparações observadas e previstas. (a) série obrigatória em escala linear; (b) todos os tamanhos em escala log-log, com a reta de referência $n^2/2$.

Grandeza	Valor
Registros lidos da origem	6.250
Nomes inseridos (m)	6.000
Nomes redundantes (d)	250
Comparações observadas	18.761.097
$m(m+1)/2$ (custo das inserções)	18.003.000
Excesso (custo das d buscas com sucesso)	758.097
Excesso médio por redundante	3032,4

Tabela 3: Execução completa sobre `bolsistas-2025.csv`.

10 Comparação entre os resultados teóricos e empíricos

Em todos os 10 tamanhos medidos, de 10 a 5.120 nomes, o número de comparações observado é *igual* ao previsto por $C(n) = n(n+1)/2$: a diferença é zero em cada linha das Tabelas 1 e 2. Os cinco valores da série obrigatória (55, 210, 820, 3.240 e 12.880) coincidem com o exemplo do enunciado.

Os indicadores assintóticos também concordam com a classificação $\Theta(n^2)$:

- $C(n)/n^2$ decresce e se aproxima de 1/2 (vale 0,5001 para $n = 5.120$);
- a razão $C(n)/C(n/2)$ observada é idêntica à teórica $4(n+1)/(n+2)$ e se aproxima de 4 (vale 3,999 no maior n);
- o coeficiente angular da reta ajustada por mínimos quadrados a $\ln C(n)$ contra $\ln n$ (Figura 1b) é 1,988, próximo do expoente 2 (fica ligeiramente abaixo de 2 por causa da parcela linear $n/2$, que perde importância quando n cresce).

Na execução completa (Tabela 3) a hipótese H2 não vale, pois o arquivo tem 250 nomes repetidos. As 6.000 inserções custam exatamente $m(m+1)/2 = 18.003.000$ comparações, porque cada inserção percorre o destino inteiro. O excesso de 758.097 comparações corresponde às buscas dos nomes repetidos, que terminam no ponto em que o nome é encontrado (em média 3032,4 comparações cada).

11 Discussão dos resultados

1. **Os dados empíricos confirmam a hipótese de complexidade?** Sim. A hipótese ($C(n) = n(n+1)/2$, logo $\Theta(n^2)$) previu exatamente todos os 10 valores medidos, em uma faixa de n que cobre mais de duas ordens de grandeza (10 a 5.120).

2. **Como se comporta a taxa de crescimento quando n dobra?** A razão observada sobe de 3,818 (10→20) a 3,999 e tende a 4, coerente com $C(2n)/C(n) = 4 - 2/(n+1)$. Um algoritmo linear daria razão 2 e um cúbico daria 8; os dados apontam inequivocamente para o regime quadrático.

3. **Há diferenças entre valores teóricos e medidos?** Não: a diferença é zero em todos os pontos. Isso é esperado porque o que se mede é uma contagem de operações, determinística, e não um tempo de execução, que teria ruído. Diferenças apareceriam se o modelo e o programa divergissem. Por exemplo: (a) se o cabeçalho não fosse comparado, o observado seria $n(n-1)/2$, ou seja, n comparações a menos; (b) com nomes repetidos, a busca bem-sucedida termina antes do fim, como na execução completa; (c) com destino previamente preenchido, cada inserção começaria de um tamanho maior que 1.

4. **Os resultados sustentam a demonstração matemática?** Sim, no sentido em que são consistentes com ela e verificam que o programa instrumentado implementa o modelo (Lema 1). Os experimentos, porém, são casos particulares: quem garante a validade para *todo* n é a indução. Os dois se complementam: a prova garante a fórmula sob H1–H3 e as medições garantem que o código executa o que as hipóteses descrevem.

5. **Qual é a evidência empírica da complexidade indicada?** Quatro evidências independentes: (i) igualdade exata entre observado e previsto em 10 tamanhos; (ii) $C(n)/n^2 \to 1/2$; (iii) razão ao dobrar $\to 4$; (iv) inclinação log-log 1,988 ≈ 2.

6. **Comparação explícita dos dados com a hipótese para $C(n)$.** A hipótese $C(n) = n(n+1)/2$ poderia ter sido enfraquecida por razões distantes de 4, por inclinação log-log longe de 2 ou por diferenças sistemáticas observado − previsto. Nenhuma ocorreu: a

hipótese sai *fortalecida*. Cabe a ressalva de que medições finitas não provam um resultado assintótico por si sós; aqui o resultado assintótico se apoia na dedução e na indução, e os dados as corroboram.

12 Conclusão

A função deduzida, $C(n) = \sum_{k=1}^{n} k = \dfrac{n(n+1)}{2}$, foi demonstrada por indução matemática (caso base $C(0) = 0$, hipótese $C(k) = k(k+1)/2$ e passo $C(k+1) = C(k) + (k+1) = (k+1)(k+2)/2$). Dela decorre que o algoritmo de importação com busca sequencial é $\Theta(n^2)$, e que dobrar o número de nomes multiplica as comparações por $4 - 2/(n+1)$, tendendo a 4.

Os experimentos com o contador instrumentado deram 55, 210, 820, 3.240 e 12.880 comparações para 10, 20, 40, 80 e 160 nomes, sem nenhuma diferença em relação ao modelo, e a série ampliada até 5.120 nomes manteve a igualdade exata e aproximou $C(n)/n^2$ de 1/2, a razão de dobra de 4 e a inclinação log-log de 2. Assim, a hipótese de complexidade quadrática é válida à luz das evidências empíricas, que reforçam a demonstração formal. Como observação de projeto, o custo quadrático vem da busca sequencial, e uma estrutura indexada (por exemplo, tabela de dispersão) reduziria o custo esperado a $O(n)$, mas sairia do escopo do algoritmo analisado.

A Código-fonte (`src/copiador.c`)

```c
/*
 * copiador.c
 *
 * Copia registros de um CSV de origem para um CSV de destino sem inserir
 * registros cujo campo Nome ja exista no destino. Conta quantas comparacoes
 * entre nomes foram feitas e grava as estatisticas em um terceiro CSV.
 *
 * Uso:   ./copiador origem.csv destino.csv estatisticas.csv
 * Saida: mensagens "ERRO: nome redundante encontrado: <nome>" na saida padrao.
 *
 * Convencoes (ver relatorio, secoes 2 e 5):
 *  - A busca por redundancia e sequencial, na ordem das linhas do destino.
 *  - Toda linha do destino participa da busca, inclusive o cabecalho
 *    ("Nome,Modalidade,Nivel,Agencia"), cujo campo Nome e "Nome".
 *  - Uma comparacao e uma chamada a strcmp entre o Nome lido da origem e o
 *    Nome de uma linha do destino.
 */
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>

#define CABECALHO       "Nome,Modalidade,Nivel,Agencia"
#define CABECALHO_NOME  "Nome"
#define CABECALHO_STATS "nomes_inseridos,comparacoes"

/* Nomes das linhas do destino, na mesma ordem em que aparecem no arquivo. */
typedef struct {
    char  **nomes;
    size_t  tamanho;
    size_t  capacidade;
} Cadastro;

/* Contador global de comparacoes entre nomes. */
static unsigned long long comparacoes = 0;

static void erro_fatal(const char *contexto, const char *caminho)
```

```
{
    fprintf(stderr, "copiador: %s: %s (%s)\n", contexto, caminho, strerror(errno));
    exit(1);
}

static void sem_memoria(void)
{
    fprintf(stderr, "copiador: memoria insuficiente\n");
    exit(1);
}

/* Remove \n e \r do fim da linha. */
static void remover_fim_de_linha(char *s)
{
    size_t n = strlen(s);
    while (n > 0 && (s[n - 1] == '\n' || s[n - 1] == '\r'))
        s[--n] = '\0';
}

/* Pula a marca BOM do UTF-8 (EF BB BF), se existir. */
static char *pular_bom(char *s)
{
    if ((unsigned char)s[0] == 0xEF && (unsigned char)s[1] == 0xBB &&
        (unsigned char)s[2] == 0xBF)
        return s + 3;
    return s;
}

/* Devolve uma copia alocada do primeiro campo (Nome) da linha CSV.
 * Aceita campo entre aspas ("SOBRENOME, NOME"). */
static char *extrair_nome(const char *linha)
{
    char *nome = malloc(strlen(linha) + 1);
    if (nome == NULL)
        sem_memoria();

    size_t k = 0;
    if (linha[0] == '"') {
        const char *p = linha + 1;
        while (*p != '\0') {
            if (*p == '"') {
                if (p[1] == '"') {      /* aspas escapadas */
                    nome[k++] = '"';
                    p += 2;
                    continue;
                }
                break;                  /* fim do campo */
            }
            nome[k++] = *p++;
        }
    } else {
        for (const char *p = linha; *p != '\0' && *p != ','; p++)
            nome[k++] = *p;
    }
    nome[k] = '\0';
    return nome;
}

/* Acrescenta um nome ao cadastro (assume a posse da string). */
static void cadastro_adicionar(Cadastro *c, char *nome)
{
    if (c->tamanho == c->capacidade) {
        size_t nova = c->capacidade ? c->capacidade * 2 : 1024;
        char **tmp = realloc(c->nomes, nova * sizeof *tmp);
        if (tmp == NULL)
            sem_memoria();
        c->nomes = tmp;
        c->capacidade = nova;
    }
    c->nomes[c->tamanho++] = nome;
}

/* Busca sequencial. Cada visita a uma linha do destino e uma comparacao. */
```

```c
static int nome_existe(const Cadastro *c, const char *nome)
{
    for (size_t i = 0; i < c->tamanho; i++) {
        comparacoes++;
        if (strcmp(c->nomes[i], nome) == 0)
            return 1;
    }
    return 0;
}

/* Le o destino (se existir) para o cadastro. Devolve 1 se a ultima linha do
 * arquivo nao termina em \n (para podermos completar antes de anexar). */
static int carregar_destino(const char *caminho, Cadastro *c)
{
    FILE *f = fopen(caminho, "r");
    if (f == NULL) {
        if (errno == ENOENT)
            return 0;
        erro_fatal("nao foi possivel abrir o destino", caminho);
    }

    char *linha = NULL;
    size_t cap = 0;
    ssize_t len;
    int sem_quebra = 0;
    int primeira = 1;

    while ((len = getline(&linha, &cap, f)) != -1) {
        sem_quebra = (len > 0 && linha[len - 1] != '\n');
        remover_fim_de_linha(linha);
        char *reg = primeira ? pular_bom(linha) : linha;
        primeira = 0;
        if (reg[0] == '\0')
            continue;
        cadastro_adicionar(c, extrair_nome(reg));
    }
    free(linha);
    fclose(f);
    return sem_quebra;
}

int main(int argc, char **argv)
{
    if (argc != 4) {
        fprintf(stderr, "Uso: %s origem.csv destino.csv estatisticas.csv\n", argv[0]);
        return 2;
    }
    const char *caminho_origem  = argv[1];
    const char *caminho_destino = argv[2];
    const char *caminho_stats   = argv[3];

    /* 1. Carrega o destino atual (cabecalho + registros ja existentes). */
    Cadastro cad = {0};
    int sem_quebra = carregar_destino(caminho_destino, &cad);

    FILE *destino = fopen(caminho_destino, "a");
    if (destino == NULL)
        erro_fatal("nao foi possivel abrir o destino para escrita", caminho_destino);

    if (cad.tamanho == 0) {             /* destino novo ou vazio: cria cabecalho */
        fprintf(destino, "%s\n", CABECALHO);
        cadastro_adicionar(&cad, strdup(CABECALHO_NOME));
        if (cad.nomes[0] == NULL)
            sem_memoria();
    } else if (sem_quebra) {
        fputc('\n', destino);
    }

    /* 2. Percorre a origem. */
    FILE *origem = fopen(caminho_origem, "r");
    if (origem == NULL)
        erro_fatal("nao foi possivel abrir a origem", caminho_origem);

```

```
    char *linha = NULL;
    size_t cap = 0;
    unsigned long long numero = 0, inseridos = 0;

    while (getline(&linha, &cap, origem) != -1) {
        numero++;
        remover_fim_de_linha(linha);
        char *reg = (numero == 1) ? pular_bom(linha) : linha;
        if (reg[0] == '\0')
            continue;

        char *nome = extrair_nome(reg);

        /* A primeira linha da origem e o cabecalho: nao e copiada. */
        if (numero == 1 && strcmp(nome, CABECALHO_NOME) == 0) {
            free(nome);
            continue;
        }

        if (nome_existe(&cad, nome)) {
            printf("ERRO: nome redundante encontrado: %s\n", nome);
            free(nome);
            continue;
        }

        if (fprintf(destino, "%s\n", reg) < 0)
            erro_fatal("falha ao escrever no destino", caminho_destino);
        cadastro_adicionar(&cad, nome);
        inseridos++;
    }
    free(linha);
    fclose(origem);
    if (fclose(destino) != 0)
        erro_fatal("falha ao fechar o destino", caminho_destino);

    /* 3. Estatisticas: uma linha por execucao (cabecalho se o arquivo e novo). */
    struct stat st;
    int novo = (stat(caminho_stats, &st) != 0 || st.st_size == 0);
    FILE *stats = fopen(caminho_stats, "a");
    if (stats == NULL)
        erro_fatal("nao foi possivel abrir o arquivo de estatisticas", caminho_stats);
    if (novo)
        fprintf(stats, "%s\n", CABECALHO_STATS);
    fprintf(stats, "%llu,%llu\n", inseridos, comparacoes);
    if (fclose(stats) != 0)
        erro_fatal("falha ao fechar o arquivo de estatisticas", caminho_stats);

    fprintf(stderr, "copiador: %llu nomes inseridos, %llu comparacoes\n",
            inseridos, comparacoes);

    for (size_t i = 0; i < cad.tamanho; i++)
        free(cad.nomes[i]);
    free(cad.nomes);
    return 0;
}
```